

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

Introduction

This report documents the steps I followed to complete the SQL assignment using DbGate Community and a provided SQLite football database. The goal of the assignment was to practice core SQL skills, starting from simple data selection and progressing to joins, grouping, aggregation, and ranking queries. All queries were executed locally, and screenshots are included to show the results and confirm that each task was completed correctly.

The assignment was structured in multiple parts. I began by exploring the database tables and writing basic SELECT statements. I then worked with ORDER BY, filtering, and different types of JOINS to combine data from multiple tables. In the later parts, I focused on aggregation using COUNT, AVG, MIN, and MAX, as well as GROUP BY and HAVING. The final sections involved more advanced queries using subqueries, common table expressions (CTEs), and window functions to rank players based on their average ratings.

During the assignment, I faced a few challenges. One difficulty was understanding why some queries returned no rows, especially when using HAVING conditions with large thresholds. Another challenge was correctly deciding when to use WHERE versus HAVING, and how aggregation affects the result set. I also needed to carefully check column names and keys when joining tables to avoid mismatches. Working through these issues helped me better understand how SQL processes data step by step.

Overall, this pre assignment helped me build confidence in writing SQL queries and understanding how relational databases work in practice.

PART 1 — Tool Setup (DbGate)

In the first step, I set up the required database tool, **DbGate Community**, which was needed for all later parts of the assignment.

I started by visiting the DbGate Community download page and selected the **Windows version** of the software. Since I was working on a Windows machine, I chose the **portable ZIP option**, which does not require installation. After downloading the file, I extracted it and launched DbGate directly from the folder.

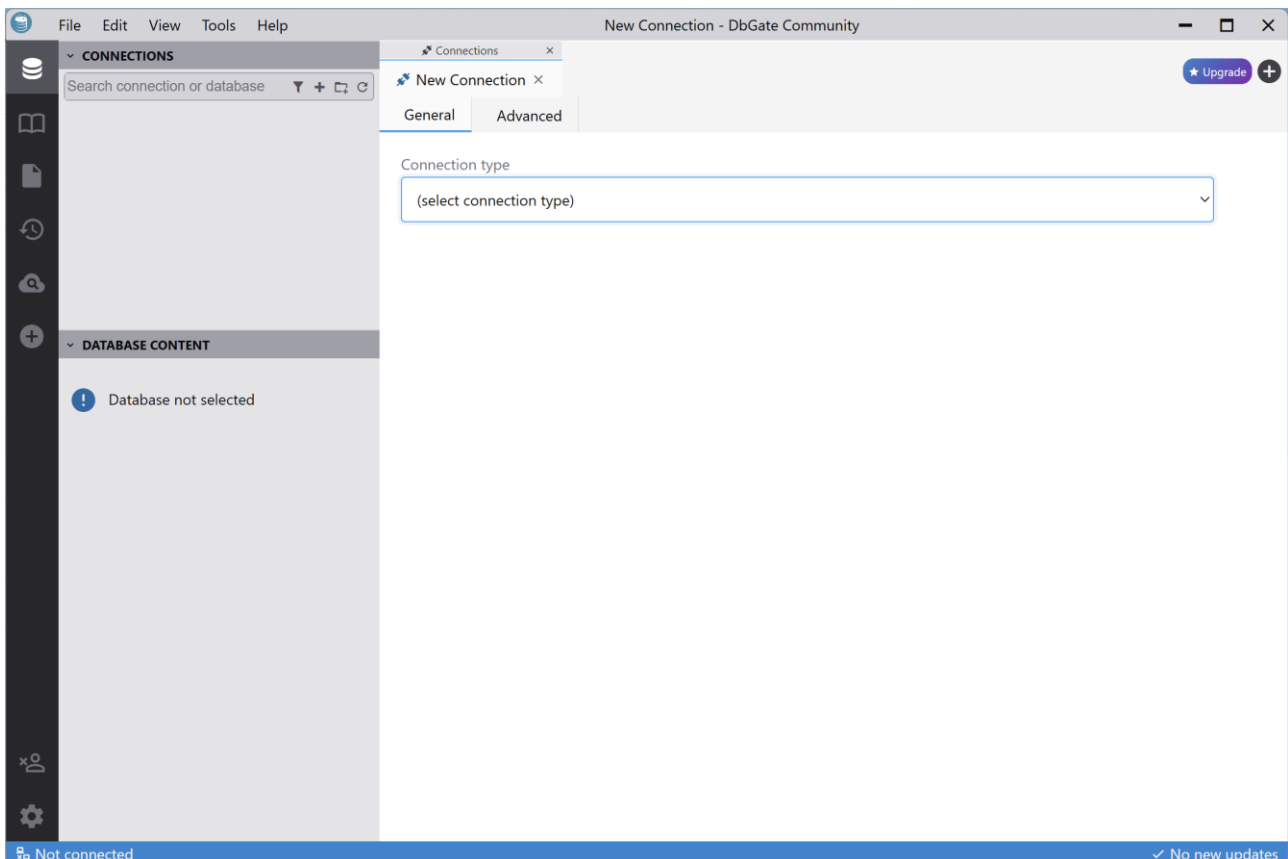
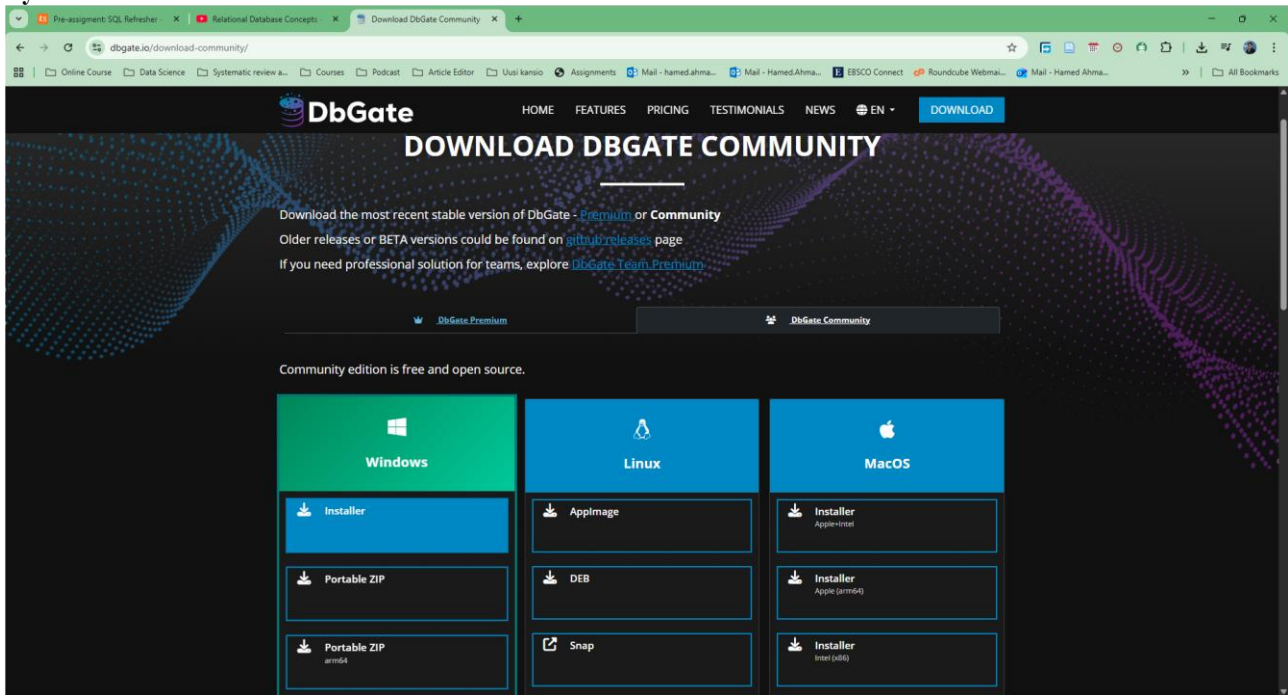
Once DbGate opened, I created a **new database connection**. In the connection settings, I selected **SQLite** as the connection type. Then, I linked the provided SQLite database file to DbGate. After saving the connection, I confirmed that it was successful by checking that the database connected without errors.

At this point, the database tables were visible in the left panel, which confirmed that DbGate was correctly set up and ready for querying. This completed the tool setup and prepared me for working with SQL queries in the following parts of the assignment.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania



Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

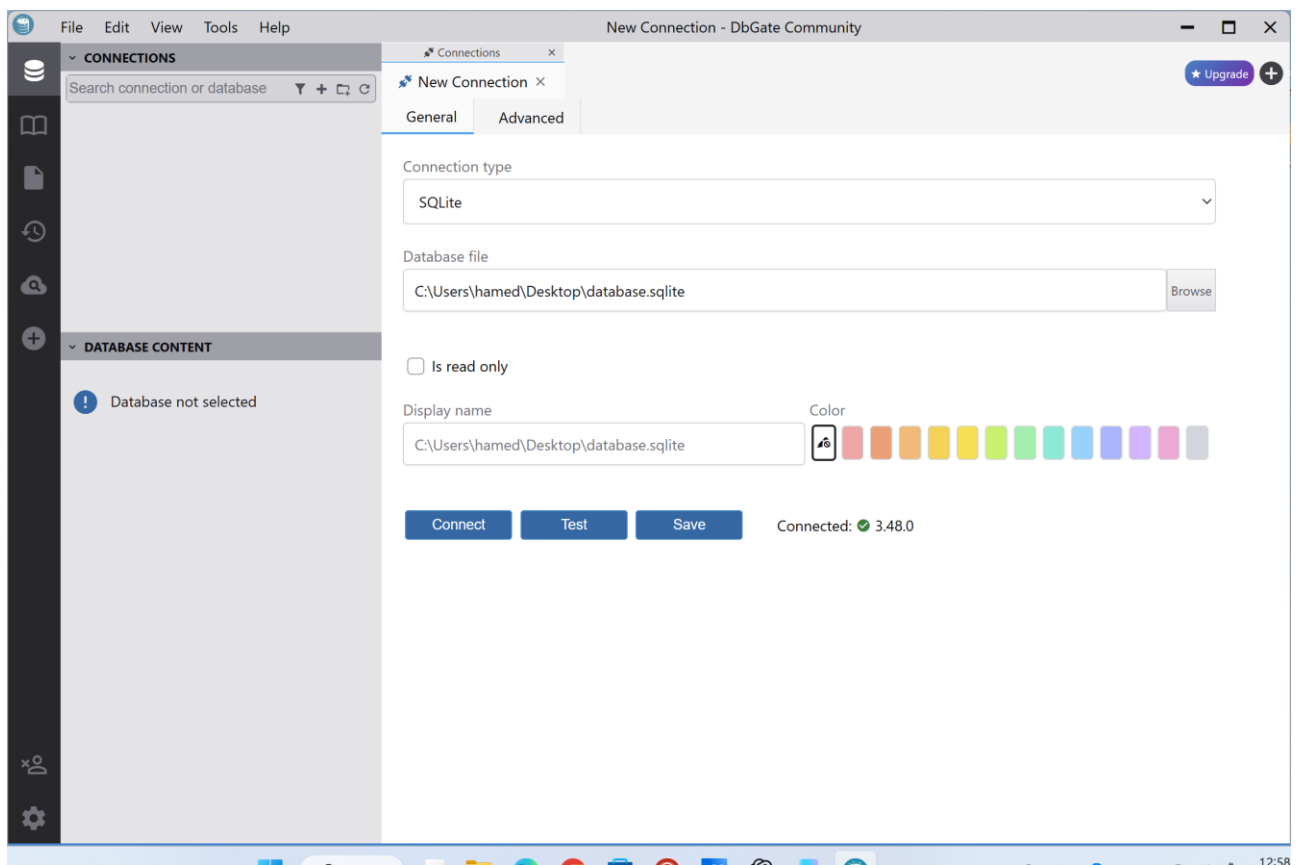
By: Hamed Ahmadiania

PART 2 — Open the Provided Database

After connecting to the SQLite database, I explored the database structure inside DbGate. In the left panel, I could see the list of available tables, including Country, League, Match, Player, Player_Attributes, Team, and Team_Attributes.

To make sure the database loaded correctly, I opened the Player table and viewed its contents. The table displayed multiple columns such as player ID, player name, birthday, height, and weight, along with thousands of rows of data. This confirmed that the database was opened successfully and that the data was accessible.

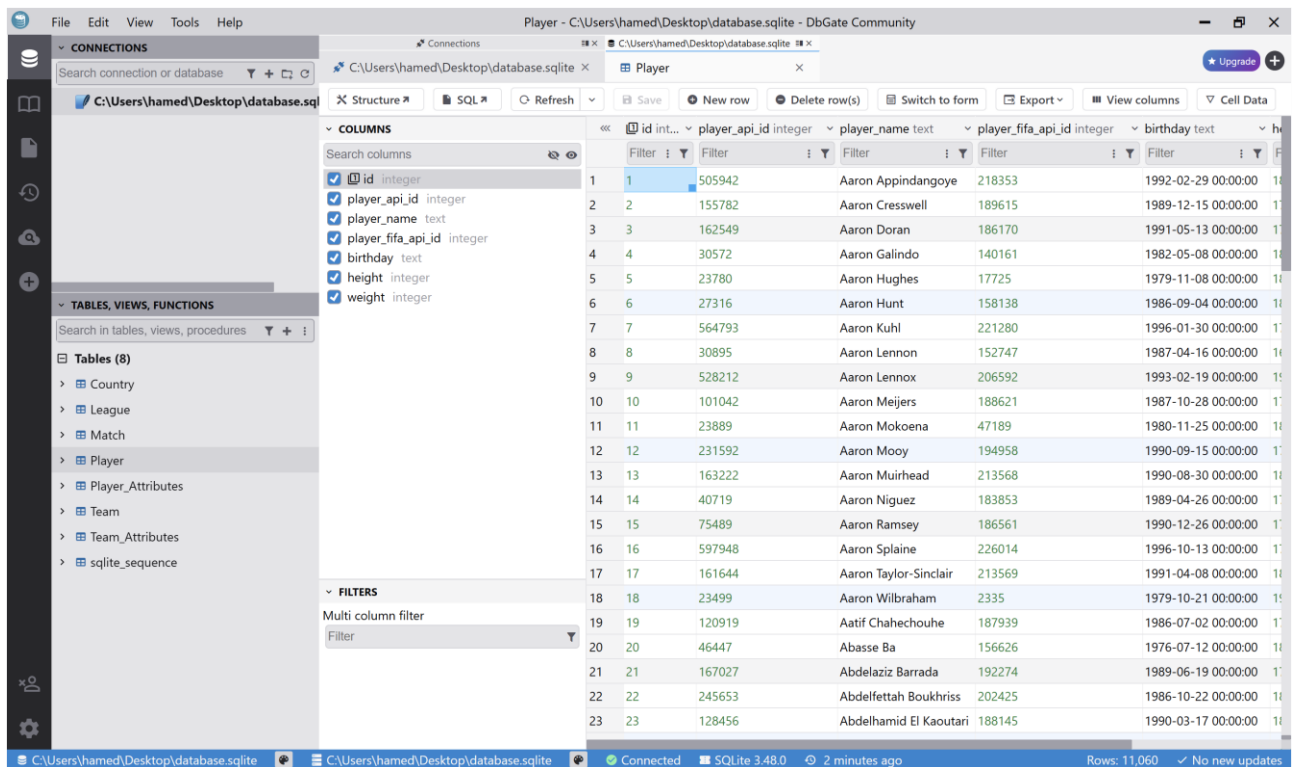
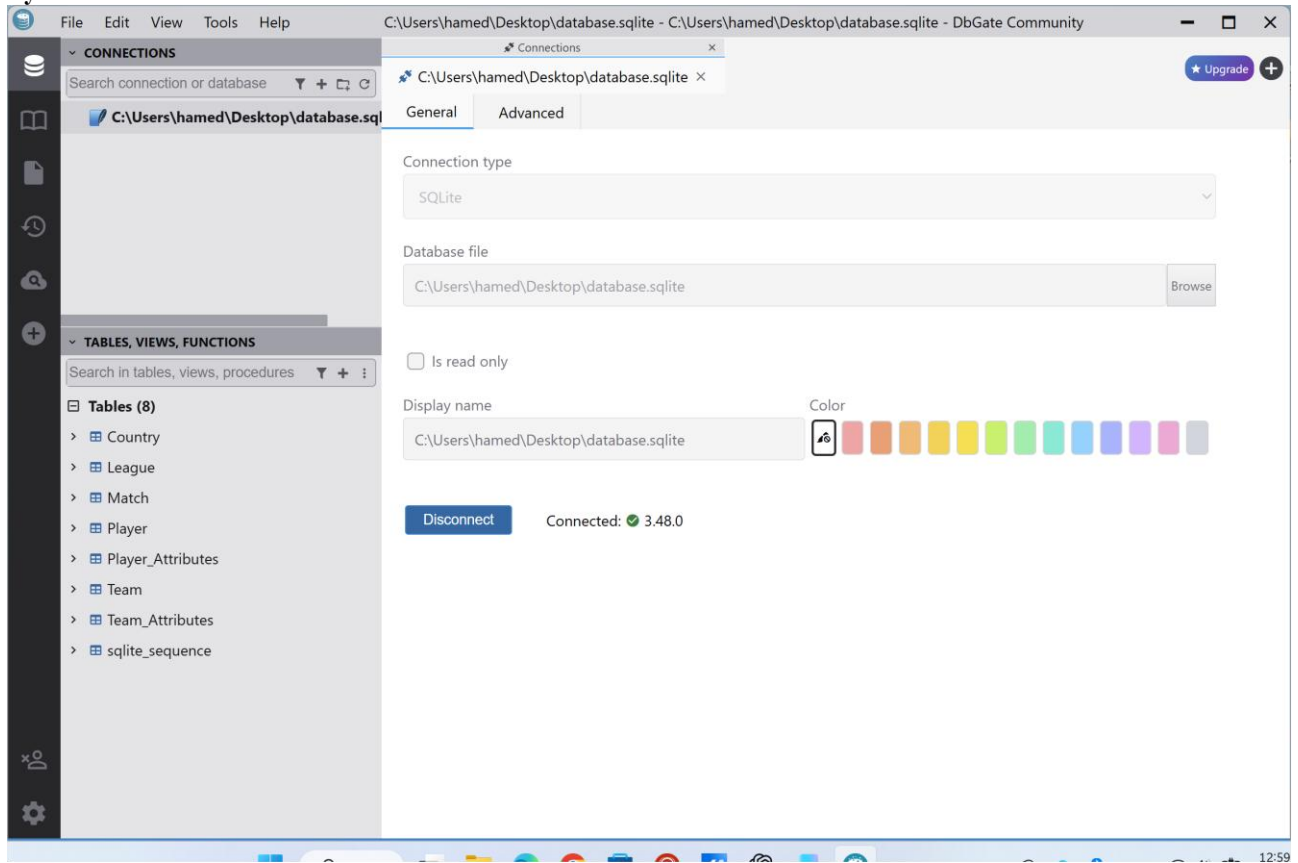
At this point, I was ready to start running SQL queries on the dataset, using the tables provided in the database.



Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania



Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

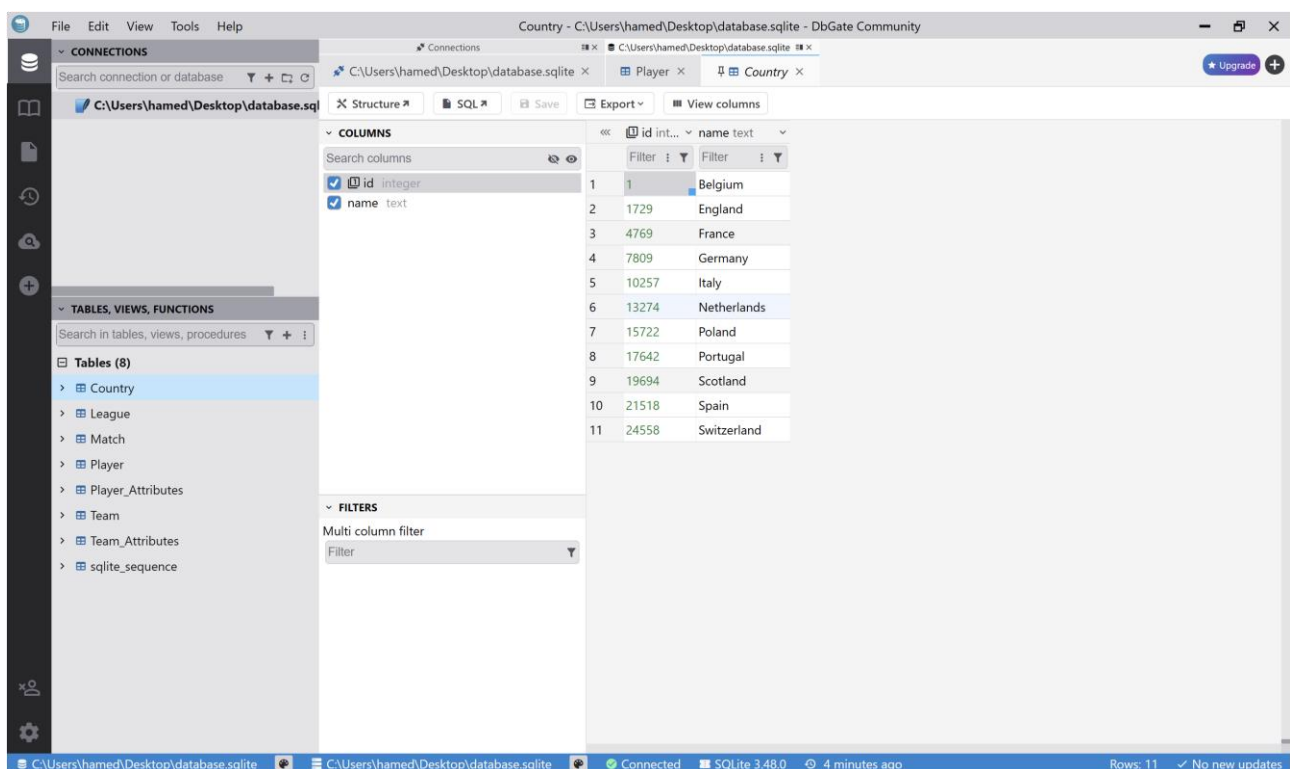
PART 3 — Relational Database Concepts

After opening the database in DbGate, I explored its structure to understand how a relational database is organized. In the left panel, I could see several tables, including Country, League, Team, Match, Player, Player_Attributes, and Team_Attributes. Each table stores a specific type of information, and together they form a relational database.

I examined the Country table first, which contains a unique ID and the country name. Then I looked at the League table and noticed that it includes a country_id column. This showed me how tables are linked: the country_id in the League table refers to the id in the Country table. This is an example of a foreign key, which connects related data across tables.

Next, I opened the Team table and observed that each team has its own identifiers and names, while being related to leagues and countries indirectly. I also reviewed the Player table, which stores individual player information such as name, birthday, height, and weight. The Player_Attributes table contains performance data and is linked to players using a shared player ID.

Through this step, I learned how relational databases avoid duplicated data by splitting information into separate tables and connecting them using keys. Understanding these relationships helped me prepare for writing SQL queries that join tables and extract meaningful information from the database.



Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface with the 'Team' table selected. The left sidebar shows the database structure with 'Team' highlighted. The main area displays the table's columns and a list of 23 rows of data.

id	int...	team_api_id	integer	team_fifa_api_id	integer	team_long_name	text	team_short_name	text
1	1	9987		673		KRC Genk		GEN	
2	2	9993		675		Beerschot AC		BAC	
3	3	10000		15005		SV Zulte-Waregem		ZUL	
4	4	9994		2007		Sporting Lokeren		LOK	
5	5	9984		1750		KSV Cercle Brugge		CEB	
6	6	8635		229		RSC Anderlecht		AND	
7	7	9991		674		KAA Gent		GEN	
8	8	9998		1747		RAEC Mons		MON	
9	9	7947		(NULL)		FCV Dender EH		DEN	
10	10	9985		232		Standard de Liège		STL	
11	11	8203		110724		KV Mechelen		MEC	
12	12	8342		231		Club Brugge KV		CLB	
13	13	9999		546		KSV Roeselare		ROS	
14	14	8571		100081		KV Kortrijk		KOR	
15	15	4049		(NULL)		Tubize		TUB	
16	16	9996		111560		Royal Excel Mouscron		MOU	
17	17	10001		681		KVC Westerlo		WES	
18	18	9986		670		Sporting Charleroi		CHA	
19	614	9997		680		Sint-Truidense VV		STT	
20	1034	9989		239		Lierse SK		LIE	
21	1042	6351		2013		KAS Eupen		EUP	
22	1513	1773		100087		Oud-Heverlee Leuven		O-H	
23	2004	8475		110913		Waasland-Beveren		WAA	

The screenshot shows the DbGate Community interface with the 'League' table selected. The left sidebar shows the database structure with 'League' highlighted. The main area displays the table's columns and a list of 11 rows of data.

id	int...	country_id	int...	name	text
1	1	1		Belgium Jupiler League	
2	1729	1729		England Premier League	
3	4769	4769		France Ligue 1	
4	7809	7809		Germany 1. Bundesliga	
5	10257	10257		Italy Serie A	
6	13274	13274		Netherlands Eredivisie	
7	15722	15722		Poland Ekstraklasa	
8	17642	17642		Portugal Liga ZON Sagres	
9	19694	19694		Scotland Premier League	
10	21518	21518		Spain LIGA BBVA	
11	24558	24558		Switzerland Super League	

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

PART 4 — Basic SQL Queries (Video 2)

In this part of the assignment, I practiced basic SQL queries using DbGate, following the instructions from the second video. The goal was to become comfortable with writing simple SELECT statements and understanding how data is retrieved from tables.

I started with very basic queries, such as selecting all columns from a table using:

```
SELECT * FROM Player;
```

This helped me confirm that the database was connected correctly and that the tables contained data. I then moved on to selecting specific columns, for example retrieving only player names, birthdays, heights, or weights. This showed me how to limit results to only the information I need instead of displaying everything.

Next, I used ORDER BY to sort results. For example, I sorted players by weight in ascending and descending order. This helped me understand how sorting works and how the ASC and DESC keywords affect the output.

I also practiced filtering data using the WHERE clause. For instance, I checked for missing values using conditions like IS NULL and IS NOT NULL. This allowed me to see which matches or records had incomplete information.

Issues I Faced and How I Fixed Them

One issue I faced early on was writing incomplete SQL statements, such as using aggregation functions like AVG() without a SELECT statement. DbGate returned errors, which helped me realize that every query must follow the correct SQL structure. I fixed this by always starting with SELECT and clearly defining the table using FROM.

Another problem I encountered was confusion between column names and table aliases, especially when I started preparing for joins later on. Sometimes queries failed because I referenced columns that did not exist in the selected table. I solved this by carefully checking column names in the table view before writing each query.

I also made mistakes with sorting and filtering, such as forgetting to include ORDER BY or placing conditions in the wrong part of the query. By reviewing error messages and comparing my queries with working examples from the video, I was able to correct these issues.

Overall, this part helped me build confidence in writing basic SQL queries and understanding how SQL retrieves, filters, and sorts data from a relational database.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with 'Tables (8)' expanded, showing 'Country', 'League', 'Match', 'Player', 'Player_Attributes', 'Team', 'Team_Attributes', and 'sqlite_sequence'. The main query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player;
```

The 'Messages' tab is selected, showing 'Result 1'. The columns selected for the query are: id, player_api_id, player_name, player_fifa_api_id, birthday, and height. The results are displayed in a table with 11,060 rows.

	id	player_api_id	player_name	player_fifa_api_id	birthday
1	1	505942	Aaron Appindangoye	218353	1992-02-29 00:00:00
2	2	155782	Aaron Cresswell	189615	1989-12-15 00:00:00
3	3	162549	Aaron Doran	186170	1991-05-13 00:00:00
4	4	30572	Aaron Galindo	140161	1982-05-08 00:00:00
5	5	23780	Aaron Hughes	17725	1979-11-08 00:00:00
6	6	27316	Aaron Hunt	158138	1986-09-04 00:00:00
7	7	564793	Aaron Kuhl	221280	1996-01-30 00:00:00
8	8	30895	Aaron Lennon	152747	1987-04-16 00:00:00
9	9	528212	Aaron Lennox	206592	1993-02-19 00:00:00

The status bar at the bottom indicates the connection is 'Connected' to 'SQLite 3.48.0' and the query was 'Finished' with 'Rows: 11,060'.

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with 'Tables (8)' expanded, showing 'Country', 'League', 'Match', 'Player', 'Player_Attributes', 'Team', 'Team_Attributes', and 'sqlite_sequence'. The main query editor contains the following SQL code:

```
1 SELECT player_name, birthday
2 FROM Player;
```

The 'Messages' tab is selected, showing 'Result 1'. The columns selected for the query are: player_name and birthday. The results are displayed in a table with 11,060 rows.

	player_name	birthday
1	Aaron Appindangoye	1992-02-29 00:00:00
2	Aaron Cresswell	1989-12-15 00:00:00
3	Aaron Doran	1991-05-13 00:00:00
4	Aaron Galindo	1982-05-08 00:00:00
5	Aaron Hughes	1979-11-08 00:00:00
6	Aaron Hunt	1986-09-04 00:00:00
7	Aaron Kuhl	1996-01-30 00:00:00
8	Aaron Lennon	1987-04-16 00:00:00
9	Aaron Lennox	1993-02-19 00:00:00

The status bar at the bottom indicates the connection is 'Connected' to 'SQLite 3.48.0' and the query was 'Finished' with 'Rows: 11,060'.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The left sidebar displays the 'CONNECTIONS' and 'TABLES, VIEWS, FUNCTIONS' panels. The main query editor shows the following SQL query:

```
1 SELECT player_name AS name
2 FROM Player;
```

The 'Messages' tab is selected, showing the 'Result 1' output. The output is a table with one column named 'name' and 11 rows of data. The status bar at the bottom indicates the query was finished, with 11,060 rows returned.

name
Aaron Appindangoye
Aaron Cresswell
Aaron Doran
Aaron Galindo
Aaron Hughes
Aaron Hunt
Aaron Kuhl
Aaron Lennon
Aaron Lennox

The screenshot shows the DbGate Community interface. The left sidebar displays the 'CONNECTIONS' and 'TABLES, VIEWS, FUNCTIONS' panels. The main query editor shows the following SQL query:

```
1 SELECT player_name AS "Full Name"
2 FROM Player;
```

The 'Messages' tab is selected, showing the 'Result 1' output. The output is a table with one column named 'Full Name' and 11 rows of data. The status bar at the bottom indicates the query was finished, with 11,060 rows returned.

Full Name
Aaron Appindangoye
Aaron Cresswell
Aaron Doran
Aaron Galindo
Aaron Hughes
Aaron Hunt
Aaron Kuhl
Aaron Lennon
Aaron Lennox

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main window shows a SQL query: `SELECT * FROM Player WHERE weight = 190;`. The results are displayed in a table with columns: id, player_api_id, player_name, player_fifa_api_id, and birthday. The status bar at the bottom indicates the connection is successful and the query is finished.

id	player_api_id	player_name	player_fifa_api_id	birthday
1	51	Abdul Aziz Tetteh	190193	1989-02-10 00:00:00
2	188	Adriano Jose de Lara	193874	1987-12-09 00:00:00
3	222	Ahmed Samir Farag	163658	1986-05-20 00:00:00
4	406	Alex Bruno	162003	1982-05-09 00:00:00
5	485	Alexandre Mendy	223510	1994-03-20 00:00:00
6	569	Alvaro Negredo	146439	1985-08-20 00:00:00
7	623	Anderson	189488	1988-01-10 00:00:00
8	666	Andre Pinto	193862	1989-10-05 00:00:00
9	685	Andrea Coda	163402	1985-04-25 00:00:00

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main window shows a SQL query: `SELECT * FROM Player WHERE weight > 190;`. The results are displayed in a table with columns: id, player_api_id, player_name, player_fifa_api_id, and birthday. The status bar at the bottom indicates the connection is successful and the query is finished.

id	player_api_id	player_name	player_fifa_api_id	birthday
1	4	Aaron Galindo	140161	1982-05-08 00:00:00
2	35	Abdoul Ba	225050	1994-02-08 00:00:00
3	45	Abdoulaye Faye	100329	1978-02-26 00:00:00
4	91	Adam Bogdan	184585	1987-09-27 00:00:00
5	103	Adam Federici	164853	1985-01-31 00:00:00
6	122	Adam Nemec	184433	1985-09-02 00:00:00
7	130	Adam Szalai	184789	1987-12-09 00:00:00
8	137	Adama Soumaoro	200353	1992-06-18 00:00:00
9	148	Adil Rami	183280	1985-12-27 00:00:00

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The SQL query executed is: `SELECT * FROM Player WHERE weight > 190;`. The result set contains 9 rows of player data.

id	player_api_id	player_name	player_fifa_api_id	birthday
1	4	Aaron Galindo	140161	1982-05-08 00:00:00
2	35	Abdoul Ba	225050	1994-02-08 00:00:00
3	45	Abdoulaye Faye	100329	1978-02-26 00:00:00
4	51	Abdul Aziz Tetteh	190193	1989-02-10 00:00:00
5	91	Adam Bogdan	184585	1987-09-27 00:00:00
6	103	Adam Federici	164853	1985-01-31 00:00:00
7	122	Adam Nemec	184433	1985-09-02 00:00:00
8	130	Adam Szalai	184789	1987-12-09 00:00:00
9	137	Adama Soumaoro	200353	1992-06-18 00:00:00

The screenshot shows the DbGate Community interface. The SQL query executed is: `SELECT * FROM Player WHERE weight > 190 AND height > 190;`. The result set contains 9 rows of player data.

id	player_api_id	player_name	player_fifa_api_id	birthday
1	35	Abdoul Ba	225050	1994-02-08 00:00:00
2	91	Adam Bogdan	184585	1987-09-27 00:00:00
3	122	Adam Nemec	184433	1985-09-02 00:00:00
4	130	Adam Szalai	184789	1987-12-09 00:00:00
5	148	Adil Rami	183280	1985-12-27 00:00:00
6	150	Adilson	200794	1982-01-01 00:00:00
7	151	Adis Jahovic	212107	1987-03-18 00:00:00
8	232	Aislan	188132	1988-01-11 00:00:00
9	246	Alain Nef	49939	1982-02-06 00:00:00

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE weight > 190 OR height > 190;
4
5
```

The query is executed, and the results are displayed in the 'Result 1' tab. The results table has the following columns: id, player_api_id, player_name, player_fifa_api_id, birthday, and height. The results are as follows:

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	4	Aaron Galindo	140161	1982-05-08 00:00:00	190
2	9	Aaron Lennox	206592	1993-02-19 00:00:00	190
3	18	Aaron Wilbraham	2335	1979-10-21 00:00:00	190
4	31	Abdeslam Ouaddou	33022	1978-11-01 00:00:00	190
5	35	Abdoul Ba	225050	1994-02-08 00:00:00	200
6	39	Abdoulaye Ba	204826	1991-01-01 00:00:00	190
7	45	Abdoulaye Faye	100329	1978-02-26 00:00:00	190
8	70	Abou Diaby	163423	1986-05-11 00:00:00	190
9	71	Aboubacar Tandia	138675	1983-10-05 00:00:00	190

The status bar at the bottom indicates: Connected, SQLite 3.48.0, 15 minutes ago, 00:00:00, Finished, Rows: 1,594, No new updates.

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE player_name = 'Aaron Galindo';
4
5
```

The query is executed, and the results are displayed in the 'Result 1' tab. The results table has the following columns: id, player_api_id, player_name, player_fifa_api_id, birthday, and height. The results are as follows:

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	4	Aaron Galindo	140161	1982-05-08 00:00:00	190

The status bar at the bottom indicates: Connected, SQLite 3.48.0, 16 minutes ago, 00:00:00, Finished, Rows: 1, No new updates.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface with a successful query execution. The query is: `SELECT * FROM Player WHERE player_name LIKE 'Aaron%';`. The result set contains 9 rows of player data.

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	505942	Aaron Appindangoye	218353	1992-02-29 00:00:00	182.88
2	155782	Aaron Cresswell	189615	1989-12-15 00:00:00	170.18
3	162549	Aaron Doran	186170	1991-05-13 00:00:00	170.18
4	30572	Aaron Galindo	140161	1982-05-08 00:00:00	182.88
5	23780	Aaron Hughes	17725	1979-11-08 00:00:00	182.88
6	27316	Aaron Hunt	158138	1986-09-04 00:00:00	182.88
7	564793	Aaron Kuhl	221280	1996-01-30 00:00:00	172.72
8	30895	Aaron Lennon	152747	1987-04-16 00:00:00	165.1
9	528212	Aaron Lennox	206592	1993-02-19 00:00:00	190.5

The screenshot shows the DbGate Community interface with a failed query execution. The query is: `SELECT * FROM Player WHERE player_name LIKE '%Aaron%';`. The error message is: "No rows loaded".

Columns: id, player_api_id, player_name, player_fifa_api_id, birthday, height

Filters: No filters applied

Result: No rows loaded

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE player_name LIKE '%Aaron%';
4
5
6
```

The query is executed, and the results are displayed in a table with 9 rows. The columns are: id, player_api_id, player_name, player_fifa_api_id, birthday, and height.

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	505942	Aaron Appindangoye	218353	1992-02-29 00:00:00	182.88
2	155782	Aaron Cresswell	189615	1989-12-15 00:00:00	170.18
3	162549	Aaron Doran	186170	1991-05-13 00:00:00	170.18
4	30572	Aaron Galindo	140161	1982-05-08 00:00:00	182.88
5	23780	Aaron Hughes	17725	1979-11-08 00:00:00	182.88
6	27316	Aaron Hunt	158138	1986-09-04 00:00:00	182.88
7	564793	Aaron Kuhl	221280	1996-01-30 00:00:00	172.72
8	30895	Aaron Lennon	152747	1987-04-16 00:00:00	165.1
9	528212	Aaron Lennox	206592	1993-02-19 00:00:00	190.5

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE player_name LIKE 'T_%';
4
5
6
```

The query is executed, and the results are displayed in a table with 9 rows. The columns are: id, player_api_id, player_name, player_fifa_api_id, birthday, and height.

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	10140	Tamas Hajnal	150360	1981-03-15 00:00:00	167
2	10141	Tamas Kadar	186648	1990-03-14 00:00:00	187
3	10142	Tamir Cohen	150229	1984-03-04 00:00:00	180
4	10143	Tamir Kahlon	169975	1987-10-29 00:00:00	177
5	10164	Temitope Ayoluwa Obadeyi	189792	1989-10-29 00:00:00	182
6	10290	Tim Bakens	148863	1982-11-02 00:00:00	195
7	10291	Tim Borowski	30630	1980-05-02 00:00:00	193
8	10292	Tim Breukers	186503	1987-11-04 00:00:00	177
9	10293	Tim Cahill	51412	1979-12-06 00:00:00	177

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE player_name IN ('Cristiano Ronaldo', 'Lionel Messi');
4
5
6
```

The query is executed, and the results are displayed in a table with columns: id, player_api_id, player_name, player_fifa_api_id, birthday, and height. The results show two rows for Cristiano Ronaldo and Lionel Messi.

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	1995	Cristiano Ronaldo	20801	1985-02-05 00:00:00	185.42
2	6176	Lionel Messi	158023	1987-06-24 00:00:00	170.18

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT *
2 FROM Player
3 WHERE weight BETWEEN 180 AND 190;
4
5
6
7
```

The query is executed, and the results are displayed in a table with columns: id, player_api_id, player_name, player_fifa_api_id, birthday, and height. The results show 9 rows of player data.

id	player_api_id	player_name	player_fifa_api_id	birthday	height
1	505942	Aaron Appindangoye	218353	1992-02-29 00:00:00	182.88
2	9	Aaron Lennox	206592	1993-02-19 00:00:00	190.5
3	11	Aaron Mokoena	47189	1980-11-25 00:00:00	182.88
4	20	Abasse Ba	156626	1976-07-12 00:00:00	187.96
5	31	Abdeslam Ouaddou	33022	1978-11-01 00:00:00	190.5
6	47	Abdoulaye Meite	41745	1980-10-06 00:00:00	185.42
7	51	Abdul Aziz Tetteh	190193	1989-02-10 00:00:00	182.88
8	71	Aboubacar Tandia	138675	1983-10-05 00:00:00	193.04
9	85	Adailton	158505	1983-04-14 00:00:00	190.5

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Match
3 WHERE home_player_1 IS NULL;
```

The results pane displays the following data:

	away_team_api_id	home_team_goal	away_team_goal	home_player_X1	home
1	9993	1	1	(NULL)	(NULL)
2	9994	0	0	(NULL)	(NULL)
3	8635	0	3	(NULL)	(NULL)
4	9998	5	0	(NULL)	(NULL)
5	9985	1	3	(NULL)	(NULL)
6	8342	1	1	(NULL)	(NULL)
7	8571	2	2	(NULL)	(NULL)
8	9996	1	2	(NULL)	(NULL)
9	9986	1	0	(NULL)	(NULL)

The status bar at the bottom indicates: Connected, SQLite 3.48.0, 25 minutes ago, 00:00:00, Finished, Rows: 1,224, No new updates.

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Match
3 WHERE home_player_1 IS NOT NULL;
```

The results pane displays the following data:

	id	country_id	league_id	season	stage	date	match_a
1	145	1	1	2008/2009	24	2009-02-28 00:00:00	493016
2	146	1	1	2008/2009	24	2009-02-27 00:00:00	493017
3	147	1	1	2008/2009	24	2009-02-28 00:00:00	493018
4	149	1	1	2008/2009	24	2009-03-01 00:00:00	493020
5	150	1	1	2008/2009	24	2009-02-28 00:00:00	493021
6	151	1	1	2008/2009	24	2009-02-28 00:00:00	493022
7	152	1	1	2008/2009	24	2009-02-28 00:00:00	493023
8	153	1	1	2008/2009	24	2009-02-28 00:00:00	493024
9	154	1	1	2008/2009	25	2009-03-08 00:00:00	493025

The status bar at the bottom indicates: Connected, SQLite 3.48.0, 25 minutes ago, 00:00:01, Finished, Rows: 24,755, No new updates.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

PART 5 — ORDER BY (Sorting Results)

In this part, I learned how to sort query results using the ORDER BY clause. I ran queries that sorted players by weight in ascending order and then in descending order using DESC. Sorting the data helped me quickly identify players with the lowest and highest values. I also learned that sorting happens **after** the data is selected and filtered.

Minor issue:

At one point, I forgot to specify DESC, so the results were sorted in ascending order by default. Adding DESC fixed the issue immediately.

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT player_name, weight
2 FROM Player
3 ORDER BY weight;
```

The query has been executed, and the results are displayed in the 'Result 1' tab. The results are sorted by weight in ascending order.

	player_name	weight
1	Juan Quero	117
2	Mariusz Rybicki	121
3	Yohandry Orozco	121
4	Caetano	123
5	Diego Buonanotte	123
6	Aldo Kalulu	126
7	Andre Horta	126
8	Fouad Rachid	126
9	Maxi Moralez	126

The status bar at the bottom indicates: Connected, SQLite 3.48.0, 27 minutes ago, 00:00:00, Finished, Rows: 11,060, No new updates.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community application interface. The main window displays a SQL query in the editor:

```
1 SELECT player_name, weight
2 FROM Player
3 ORDER BY weight DESC;
```

The query has been executed, and the results are shown in the 'Result 1' tab. The results are displayed in a table with two columns: 'player_name' and 'weight'.

	player_name	weight
1	Kristof van Hout	243
2	Tim Wiese	243
3	Jeroen Verhoeven	227
4	Ishmael Miller	225
5	Cassio	220
6	Christopher Samba	220
7	Lars Unnerstall	220
8	Marcus Hahnemann	220
9	Abdoulaye Faye	218

The interface also shows a sidebar with 'CONNECTIONS' and 'TABLES, VIEWS, FUNCTIONS' sections. The 'TABLES, VIEWS, FUNCTIONS' section is expanded, showing a list of tables: Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The 'Player' table is selected.

The status bar at the bottom indicates the connection is 'Connected' to 'SQLite 3.48.0' and shows the query execution time as '00:00:00' and the number of rows as 'Rows: 11,060'.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

PART 6 — JOINS (Core Relational SQL)

Finally, I used JOIN statements to combine data from multiple tables. I joined tables such as **League** with **Country**, and **Match** with **League** and **Country**, using the correct foreign keys. This allowed me to display meaningful combined information, such as league names with their countries and match details with league and season information.

Issues faced and fixes:

- Initially, some join queries failed because I joined tables using the wrong columns. I fixed this by carefully matching foreign keys (for example, league_id with id).
- I also learned that table aliases help make join queries clearer and easier to read.

The screenshot shows the DbGate Community interface. The main window displays a SQL query for Query #1:

```
1 SELECT
2   p.player_name,
3   t.team_long_name
4 FROM Player p
5 JOIN Team t
6   ON p.team_id = t.team_api_id;
```

The left sidebar shows the database structure with tables: Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence.

The bottom panel shows the execution messages:

Number	Message	Time	Delta	Duration	Procedure	Line
1	Query execution started SELECT * FROM Player;	13:08:00	0	n/a		
2	Query execution finished	13:08:00	326 ms	326 ms		
3	Query returned 11060 rows in 67 ms	13:08:00	362 ms	36 ms		
4	Query execution started SELECT player_name, birthday FROM Player;	13:08:47	47 s	46 s		
5	Query execution finished	13:08:47	47 s	22 ms		
6	Query returned 11060 rows in 52 ms	13:08:47	47 s	31 ms		
7	Query execution started SELECT player_name AS name FROM Player;	13:10:04	124 s	78 s		
8	Query execution finished	13:10:04	124 s	15 ms		

The status bar at the bottom indicates the database is connected, SQLite 3.48.0, and the query was finished with no new updates.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables: Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT
2   l.name AS league_name,
3   c.name AS country_name
4 FROM League l
5 JOIN Country c
6   ON l.country_id = c.id;
```

The query results are displayed in a table with columns 'league_name' and 'country_name'.

league_name	country_name
Belgium Jupiler League	Belgium
England Premier League	England
France Ligue 1	France
Germany 1. Bundesliga	Germany
Italy Serie A	Italy
Netherlands Eredivisie	Netherlands
Poland Ekstraklasa	Poland
Portugal Liga ZON Sagres	Portugal
Scotland Premier League	Scotland

The status bar at the bottom indicates the connection is successful, using SQLite 3.48.0, and the query finished with 11 rows.

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure with tables: Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT
2   m.id AS match_id,
3   l.name AS league_name,
4   m.season
5 FROM Match m
6 JOIN League l
7   ON m.league_id = l.id;
```

The query results are displayed in a table with columns 'match_id', 'league_name', and 'season'.

match_id	league_name	season
1	Belgium Jupiler League	2008/2009
2	Belgium Jupiler League	2008/2009
3	Belgium Jupiler League	2008/2009
4	Belgium Jupiler League	2008/2009
5	Belgium Jupiler League	2008/2009
6	Belgium Jupiler League	2008/2009
7	Belgium Jupiler League	2008/2009
8	Belgium Jupiler League	2008/2009
9	Belgium Jupiler League	2008/2009

The status bar at the bottom indicates the connection is successful, using SQLite 3.48.0, and the query finished with 25,979 rows.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

The screenshot shows the DbGate Community interface. The main window displays a SQL query in the editor, which is a JOIN query between Match, League, and Country tables. The query is as follows:

```
1 SELECT
2   m.id AS match_id,
3   c.name AS country_name,
4   l.name AS league_name,
5   m.season
6 FROM Match m
7 JOIN League l
8   ON m.league_id = l.id
9 JOIN Country c
10    ON l.country_id = c.id;
```

The query has been executed, and the results are displayed in the 'Result 1' tab. The results show 9 rows of data, all from Belgium.

match_id	country_name	league_name	season
1	Belgium	Belgium Jupiler League	2008/2009
2	Belgium	Belgium Jupiler League	2008/2009
3	Belgium	Belgium Jupiler League	2008/2009
4	Belgium	Belgium Jupiler League	2008/2009
5	Belgium	Belgium Jupiler League	2008/2009
6	Belgium	Belgium Jupiler League	2008/2009
7	Belgium	Belgium Jupiler League	2008/2009
8	Belgium	Belgium Jupiler League	2008/2009
9	Belgium	Belgium Jupiler League	2008/2009

The interface also shows a sidebar with 'CONNECTIONS' and 'TABLES, VIEWS, FUNCTIONS'. The 'TABLES, VIEWS, FUNCTIONS' section lists 8 tables: Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence.

PART 7 — Aggregation & Grouping

In this part, I worked with aggregation functions and GROUP BY to summarize data instead of showing individual rows. I used functions like COUNT() and AVG() to calculate totals and averages from the data.

First, I counted how many players exist in the database to confirm the dataset size. Then, I grouped players by attributes such as nationality to see how many players belong to each group. This helped me understand how grouping combines many rows into a single summarized result.

I also learned that when using GROUP BY, every selected column must either be included in the GROUP BY clause or be part of an aggregate function. At first, I received an error because I selected a column that was not grouped. I fixed this by adjusting the query to follow the correct grouping rules.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure for 'database.sqlite', including tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT COUNT(*) AS total_players
2 FROM Player;
```

The 'Messages' tab is selected, showing the execution results. The 'Result 1' tab displays a single row with the column 'total_players' and the value '11060'.

total_players
11060

The status bar at the bottom indicates the connection is successful and the query is finished.

The screenshot shows the DbGate Community interface with a failed SQL query. The query editor contains the following SQL code:

```
1 SELECT nationality, COUNT(*) AS player_count
2 FROM Player
3 GROUP BY nationality
4 ORDER BY player_count DESC;
```

The 'Messages' tab is selected, showing the execution results. The 'Number Message' tab displays a list of messages, including the error message 'Query execution finished'.

Number Message	Time	Delta	Duration	Procedure	Line
1 Query execution started SELECT * FROM Player;	13:08:00	0	n/a		
2 Query execution finished	13:08:00	326 ms	326 ms		
3 Query returned 11060 rows in 67 ms Show Data	13:08:00	362 ms	36 ms		
4 Query execution started SELECT player_name, birthday FROM Player;	13:08:47	47 s	46 s		
5 Query execution finished	13:08:47	47 s	22 ms		
6 Query returned 11060 rows in 52 ms Show Data	13:08:47	47 s	31 ms		
7 Query execution started SELECT player_name AS name FROM Player;	13:10:04	124 s	78 s		

The status bar at the bottom indicates the connection is successful and the query is finished.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT height, COUNT(*) AS player_count
2 FROM Player
3 GROUP BY height
4 ORDER BY player_count DESC;
```

The results pane displays the following data:

height	player_count
182.88	1954
177.8	1489
180.34	1388
187.96	1305
185.42	1278
175.26	1188
190.5	652
172.72	530
193.04	470

The screenshot shows the DbGate Community interface with a filtered SQL query. The query editor contains the following SQL code:

```
1 SELECT height, COUNT(*) AS player_count
2 FROM Player
3 WHERE height BETWEEN 180 AND 190
4 GROUP BY height
5 ORDER BY height;
```

The results pane displays the following data:

height	player_count
180.34	1388
182.88	1954
185.42	1278
187.96	1305

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT AVG(weight) AS avg_weight
2 FROM Player;
```

The query has been executed, and the results are displayed in the 'Result 1' tab. The results table has one column, 'avg_weight', and one row with the value 168.38028933092224.

avg_weight
168.38028933092224

The screenshot shows the DbGate Community interface. The query editor contains the following SQL code:

```
1 SELECT
2 MIN(weight) AS min_weight,
3 MAX(weight) AS max_weight
4 FROM Player;
```

The query has been executed, and the results are displayed in the 'Result 1' tab. The results table has two columns, 'min_weight' and 'max_weight', and one row with the values 117 and 243.

min_weight	max_weight
117	243

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

Query #1 - C:\Users\hamed\Desktop\database.sqlite - DbGate Community

```
1 SELECT
2   l.name AS league_name,
3   COUNT(*) AS match_count
4 FROM Match m
5 JOIN League l
6   ON m.league_id = l.id
7 GROUP BY l.name
8 ORDER BY match_count DESC;
```

Result 1

	league_name	match_count
1	Spain LIGA BBVA	3040
2	France Ligue 1	3040
3	England Premier League	3040
4	Italy Serie A	3017
5	Netherlands Eredivisie	2448
6	Germany 1. Bundesliga	2448
7	Portugal Liga ZON Sagres	2052
8	Poland Ekstraklasa	1920
9	Scotland Premier League	1824

Columns: league_name, match_count

Filters: Multi column filter

Rows: 11

Query #1 - C:\Users\hamed\Desktop\database.sqlite - DbGate Community

```
1 SELECT
2   l.name AS league_name,
3   COUNT(*) AS match_count
4 FROM Match m
5 JOIN League l
6   ON m.league_id = l.id
7 GROUP BY l.name
8 HAVING COUNT(*) > 10000
9 ORDER BY match_count DESC;
```

Result 1

No rows loaded

Columns: league_name, match_count

Filters: Multi column filter

Rows: 0

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

PART 8 — Subqueries & Advanced Joins

In this part, I worked with subqueries and more advanced uses of JOINS to create more meaningful results from the database. Instead of using fixed values, I used subqueries to calculate values dynamically inside the main query.

For example, I used a subquery to calculate the overall average player rating and then filtered players whose average rating was higher than that value. This made the query more flexible and accurate, because it automatically adapts to the data in the database.

I also combined subqueries with joins between the Player and Player_Attributes tables. This allowed me to link player names with their calculated average ratings, even though the ratings were stored in a different table.

At first, I found it confusing to understand where the subquery should be placed and how it interacts with HAVING. After testing and adjusting the query structure, I was able to correctly filter grouped results using the subquery. This part helped me better understand how complex SQL queries are built step by step.

The screenshot shows the DbGate Community interface. The main window displays an SQL query for Query #1, which calculates the average rating for each player and filters for players with a rating higher than the overall average. The query is as follows:

```
1 SELECT
2   S.player_name,
3   AVG(A.overall_rating) AS rating
4 FROM player_attributes A
5 JOIN player B
6   ON A.player_api_id = B.player_api_id
7 GROUP BY
8   S.player_name
9 HAVING
10  AVG(A.overall_rating) > 85
11 ORDER BY
12   rating DESC;
```

The results are shown in a table with two columns: player_name and rating. The table lists the top 9 players by rating.

player_name	rating
Lionel Messi	92.1923076923077
Cristiano Ronaldo	91.28
Franck Ribery	88.45833333333333
Andres Iniesta	88.32
Zlatan Ibrahimovic	88.28571428571429
Arjen Robben	87.84
Xavi Hernandez	87.63636363636364
Wayne Rooney	87.22222222222223
Iker Casillas	86.95454545454545

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The left sidebar displays the database structure for 'database.sqlite', including tables like Country, League, Match, Player, Player_Attributes, Team, Team_Attributes, and sqlite_sequence. The main query editor contains the following SQL code:

```
1 SELECT
2     B.player_name,
3     ROUND(AVG(A.overall_rating), 2) AS rating
4 FROM player_attributes A
5 JOIN player B
6     ON A.player_api_id = B.player_api_id
7 GROUP BY
8     B.player_name
9 HAVING
10    AVG(A.overall_rating) > 85
11 ORDER BY
12    rating DESC;
```

The 'Result 1' tab shows the query results in a table with columns 'player_name' and 'rating'. The results are as follows:

player_name	rating
Lionel Messi	92.19
Cristiano Ronaldo	91.28
Franck Ribery	88.46
Andres Iniesta	88.32
Zlatan Ibrahimovic	88.29
Arjen Robben	87.84
Xavi Hernandez	87.64
Wayne Rooney	87.22
Iker Casillas	86.95

The status bar at the bottom indicates the connection is successful, using SQLite 3.48.0, and the query was finished in 00:00:00 with 26 rows returned.

The screenshot shows the DbGate Community interface with a more complex SQL query. The query editor contains the following SQL code:

```
1 SELECT
2     B.player_name,
3     ROUND(AVG(A.overall_rating), 2) AS rating
4 FROM player_attributes A
5 JOIN player B
6     ON A.player_api_id = B.player_api_id
7 GROUP BY
8     B.player_name
9 HAVING
10    AVG(A.overall_rating) >
11    (
12        SELECT AVG(overall_rating)
13        FROM player_attributes
14    )
15 ORDER BY
16    rating DESC;
```

The 'Result 1' tab shows the query results in a table with columns 'player_name' and 'rating'. The results are identical to the first screenshot:

player_name	rating
Lionel Messi	92.19
Cristiano Ronaldo	91.28
Franck Ribery	88.46
Andres Iniesta	88.32
Zlatan Ibrahimovic	88.29
Arjen Robben	87.84
Xavi Hernandez	87.64
Wayne Rooney	87.22
Iker Casillas	86.95

The status bar at the bottom indicates the connection is successful, using SQLite 3.48.0, and the query was finished in 00:00:00 with 4,108 rows returned.

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

PART 10 — Advanced Querying (CTEs + Window Functions)

In this part, I used Common Table Expressions (CTEs) and window functions to write more advanced and well-structured SQL queries. I first created a CTE to calculate the average overall rating for each player from the Player_Attributes table. This made the query easier to read and avoided repeating the same calculation multiple times.

After that, I applied window functions such as DENSE_RANK() to rank players based on their average ratings. Instead of collapsing the data like GROUP BY, the window function allowed me to keep individual rows while still adding ranking information.

I then joined the ranked results with the Player table to display player names alongside their rankings and rounded average ratings. Using WHERE conditions on the ranking column, I was able to select only the top players.

The screenshot shows the DbGate Community interface with a query editor and a results pane. The query uses a CTE to calculate the average rating for each player and then ranks them using DENSE_RANK(). The results pane shows the top 9 players by average rating.

```
1 WITH player_avg AS (  
2     SELECT  
3         player_api_id,  
4         AVG(overall_rating) AS avg_rating  
5     FROM Player_Attributes  
6     GROUP BY player_api_id  
7 )  
8 SELECT  
9     p.player_name,  
10    ROUND(a.avg_rating, 2) AS avg_rating  
11 FROM player_avg a  
12 JOIN Player p  
13 ON p.player_api_id = a.player_api_id  
14 ORDER BY avg_rating DESC;
```

	player_name	avg_rating
1	Lionel Messi	92.19
2	Cristiano Ronaldo	91.28
3	Franck Ribery	88.46
4	Andres Iniesta	88.32
5	Zlatan Ibrahimovic	88.29
6	Arjen Robben	87.84
7	Xavi Hernandez	87.64
8	Wayne Rooney	87.22
9	Iker Casillas	86.95

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface with a SQLite database. The query executed is as follows:

```
1 WITH player_avg AS (  
2   SELECT  
3     player_api_id,  
4     AVG(overall_rating) AS avg_rating  
5   FROM Player_Attributes  
6   GROUP BY player_api_id  
7 ),  
8 ranked AS (  
9   SELECT  
10    player_api_id,  
11    avg_rating,  
12    DENSE_RANK() OVER (ORDER BY avg_rating DESC) AS rnk  
13  FROM player_avg  
14 )  
15 SELECT  
16   p.player_name,  
17   round(r.avg_rating, 2) AS avg_rating,  
18   r.rnk  
19 FROM ranked r  
20 JOIN Player p  
21 ON p.player_api_id = r.player_api_id  
22 WHERE r.rnk <= 5  
23 ORDER BY r.rnk, p.player_name;
```

The results table shows the top 5 players by average rating:

player_name	avg_rating	rnk
Lionel Messi	92.19	1
Cristiano Ronaldo	91.28	2
Franck Ribery	88.46	3
Andres Iniesta	88.32	4
Zlatan Ibrahimovic	88.29	5

The screenshot shows the same DbGate Community interface, but the query has been modified to show the top 10 players. The query is:

```
1 WITH player_avg AS (  
2   SELECT  
3     player_api_id,  
4     AVG(overall_rating) AS avg_rating  
5   FROM Player_Attributes  
6   GROUP BY player_api_id  
7 ),  
8 ranked AS (  
9   SELECT  
10    player_api_id,  
11    avg_rating,  
12    DENSE_RANK() OVER (ORDER BY avg_rating DESC) AS rnk  
13  FROM player_avg  
14 )  
15 SELECT  
16   p.player_name,  
17   round(r.avg_rating, 2) AS avg_rating,  
18   r.rnk  
19 FROM ranked r  
20 JOIN Player p  
21 ON p.player_api_id = r.player_api_id  
22 WHERE r.rnk <= 10  
23 ORDER BY r.rnk;
```

The results table now shows the top 10 players:

player_name	avg_rating	rnk
Lionel Messi	92.19	1
Cristiano Ronaldo	91.28	2
Franck Ribery	88.46	3
Andres Iniesta	88.32	4
Zlatan Ibrahimovic	88.29	5
Arjen Robben	87.84	6

Assignment Pre-assignment: SQL Refresher

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania

Conclusion

In this assignment, I successfully completed all required steps, starting from setting up DbGate and opening the provided SQLite database to writing basic and advanced SQL queries. Through each part, I learned how relational databases are structured and how tables are connected using keys and joins. I practiced selecting, filtering, sorting, grouping, and aggregating data, and later applied more advanced techniques such as subqueries, CTEs, and window functions. Along the way, I faced small challenges, mainly related to query syntax, joins, and understanding how grouping and filtering work together, but testing queries step by step helped me fix these issues. Overall, this assignment helped me build confidence in using SQL and better understand how to work with real relational databases in a practical way.